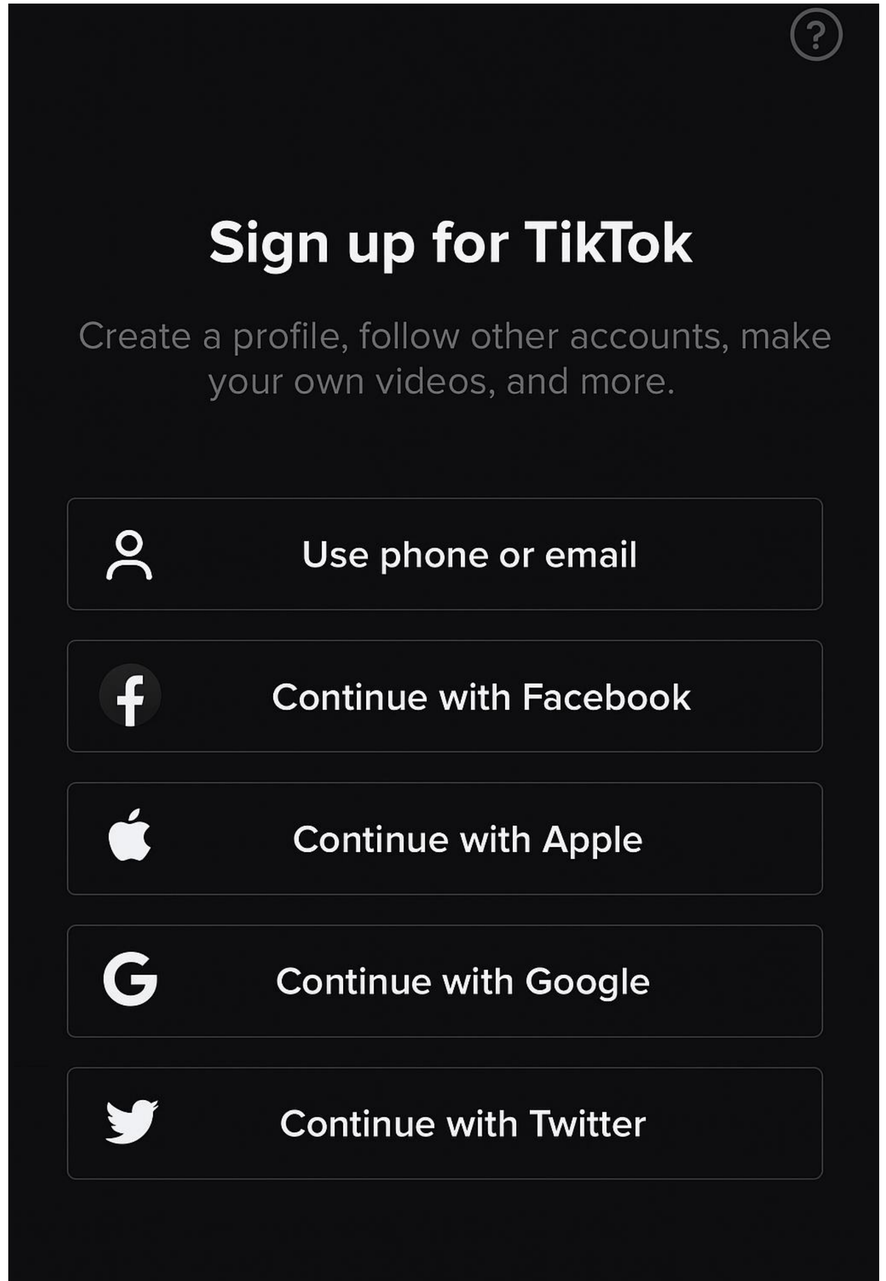
TikTok Audio Popularity Prediction

Aishwarya Seshadri



Agenda

1. Topic
2. Dataset
3. Exploration
4. Limitations
5. Insights

Overview

Question:

What factors drive audio virality on TikTok?

Possible Applications:

Song marketing, music promotion, and boosted content creation



Topic — Dataset — Exploration — Limitations — Insights

Dataset Overview

Background

- Around 7000 trending TikTok tracks with variables
- Published by Yam Peleg on Kaggle

23 Columns

6746 Rows

- **Cut 12 repetitive variables**
- **Removed rows with null values**
- **Left with 11 columns and 3196 rows**

Dataset Overview

Basic Info

Variables
Title, Artist, Year, Popularity, Danceability, Energy, Tempo, Duration, Valence, Liveness, Loudness

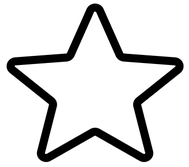
Song Selection
Popular TikTok audios from 1960-2021

Popularity Index

What Is It?
A numeric score from 1-100 measuring TikTok virality

Determined By
TikTok usage frequency, engagement metrics (likes, shares), music popularity

Quantitative Song Characteristics



Popularity
(0-100)



Danceability
(0.0-1.0)



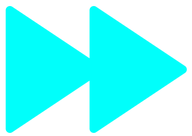
Energy



Liveness



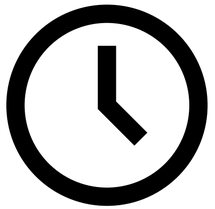
Valence



Tempo
(BPM)

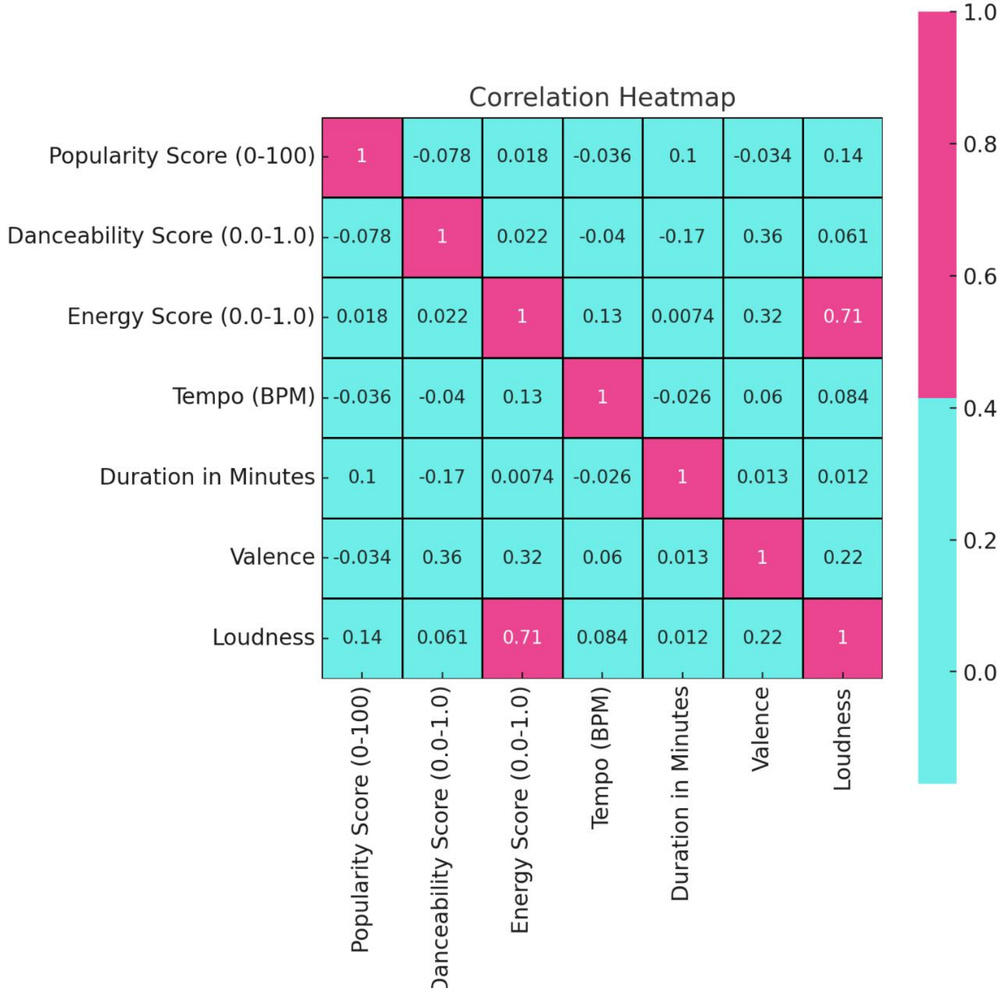


Loudness
(decibels)



Duration
(min)

Correlation Heat Map



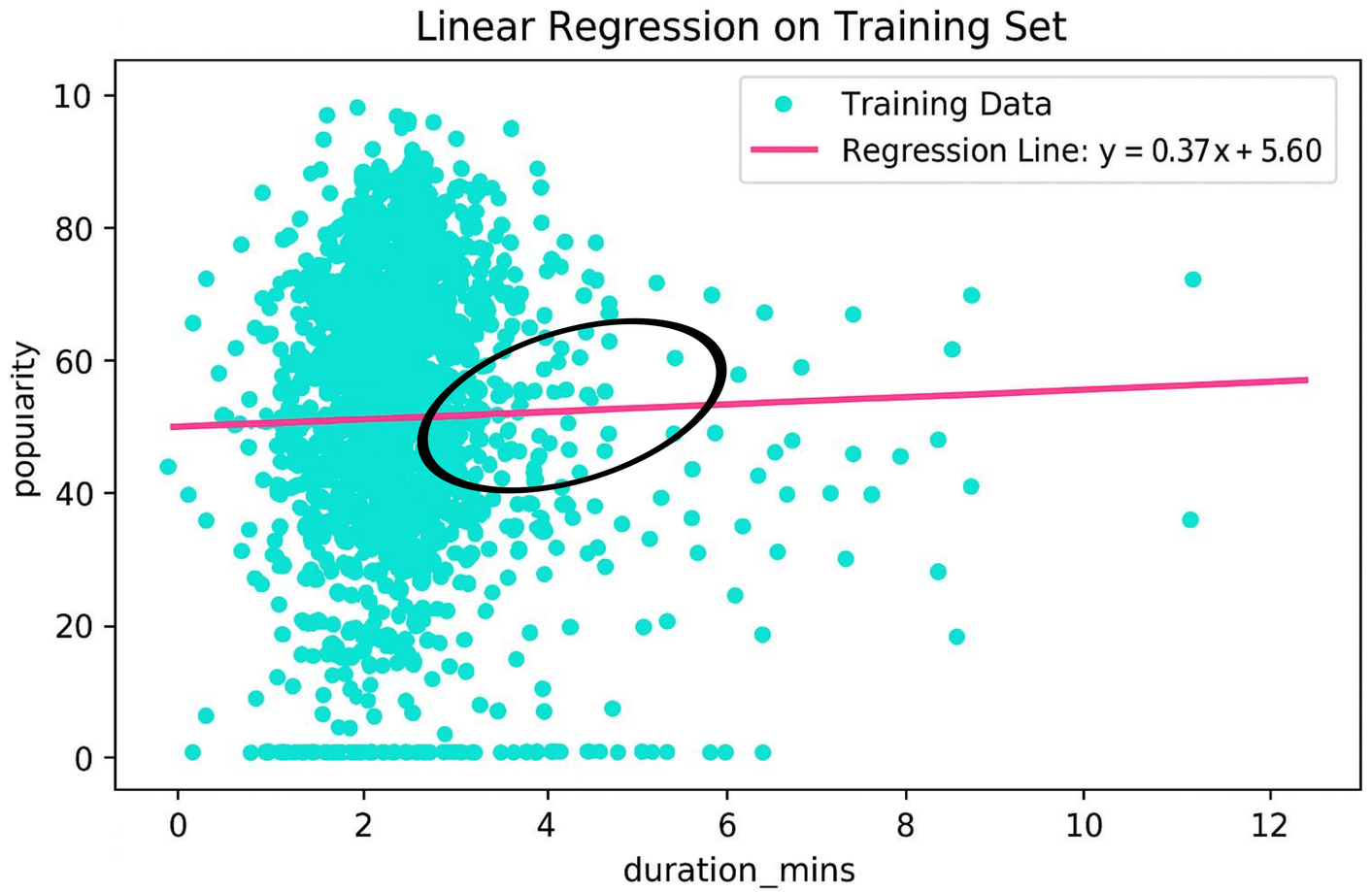
Observations:

- Loudness & Energy - strong positive correlation (r=0.71)
- Danceability & Valence - moderate positive correlation (r=0.36)
- Loudness & Popularity - mild positive correlation (r=0.14)
- Danceability, Energy, & Valence have a weak but positive correlation with Popularity
- Tempo & Duration have little to no corelation with Popularity
- No variables show a strong linear correlation with Popularity!

Regression Models

Linear Regression

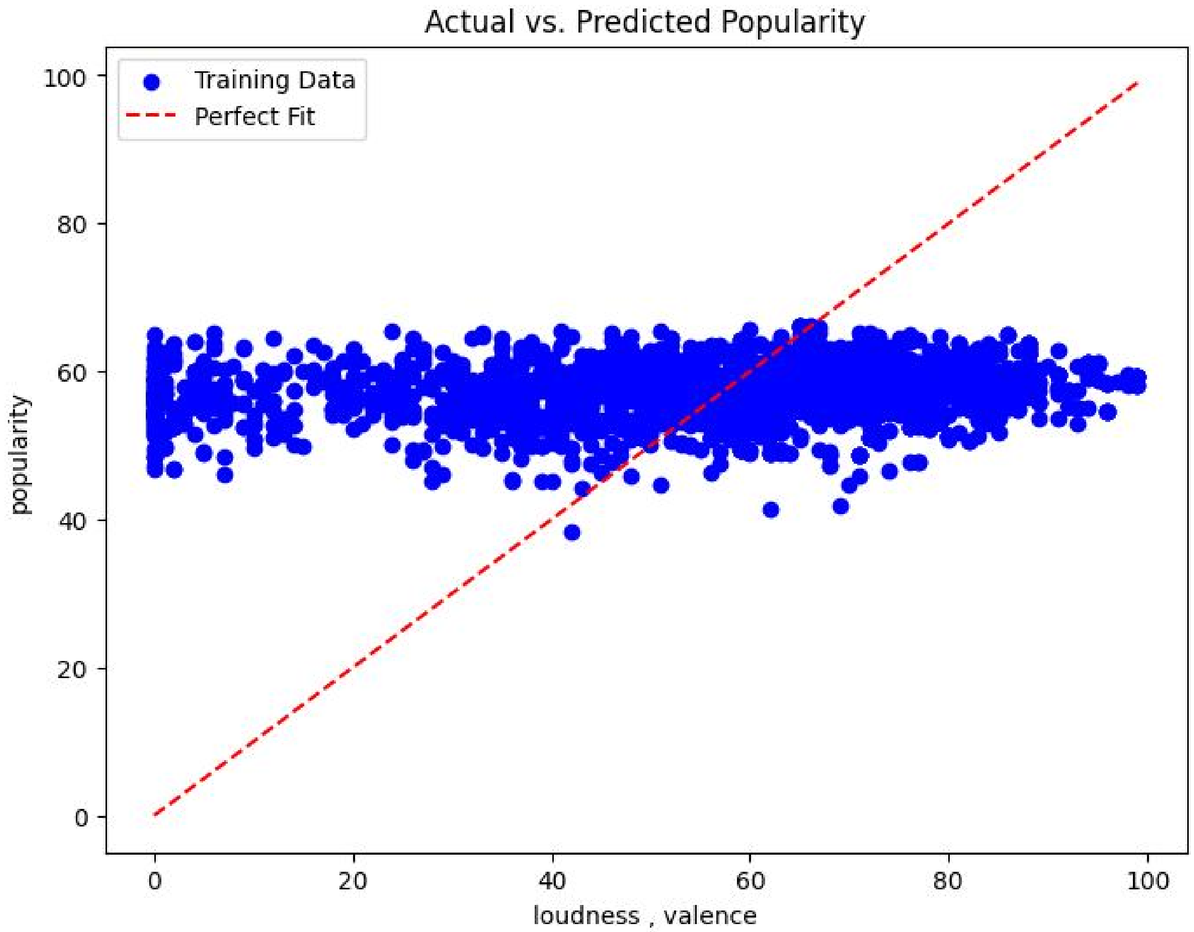
R2: -0.00
MSE: 0.02



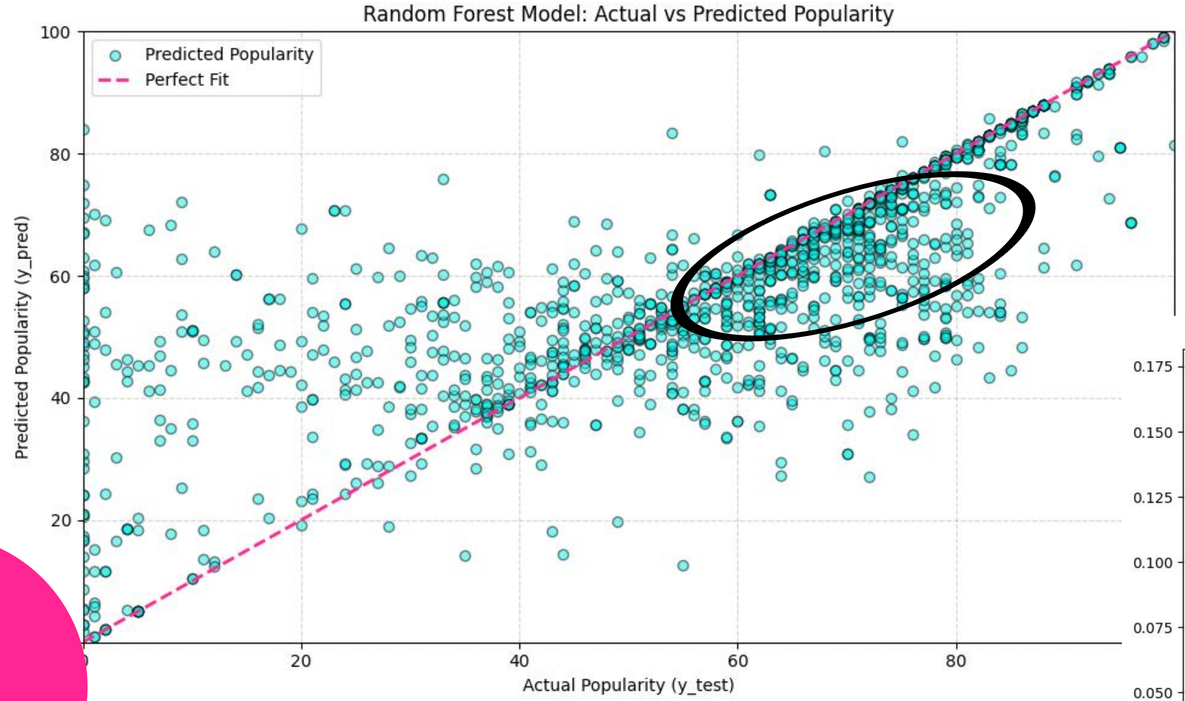
Regression Models

Multi-Linear Regression

R2: 0.03
MSE: 557.5

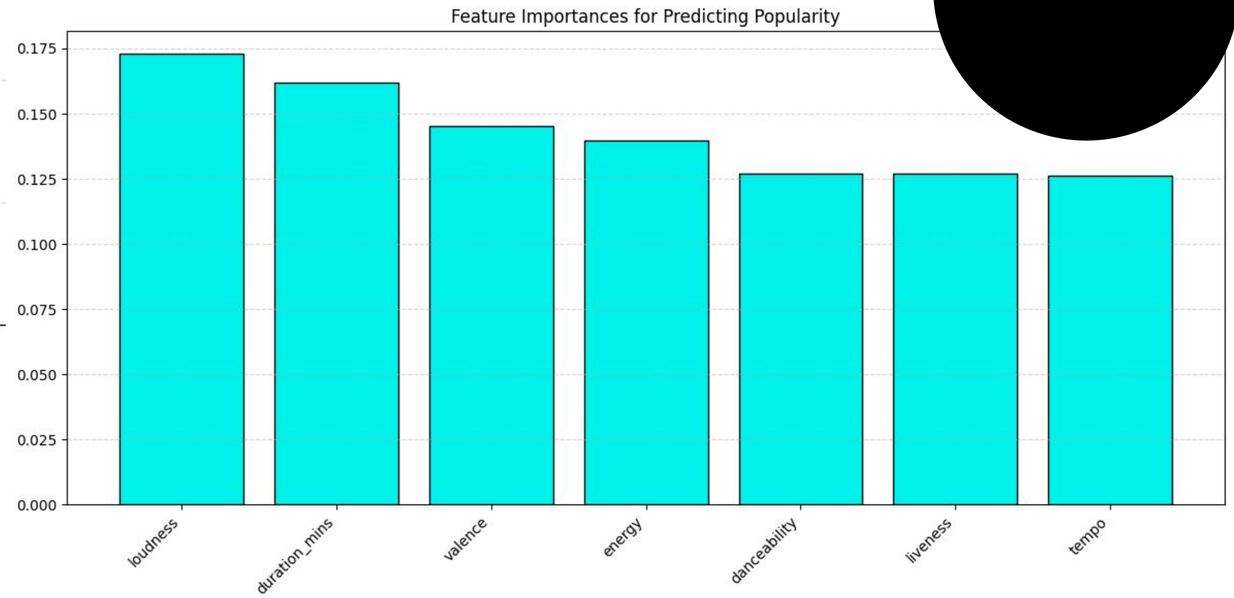


Regression Models



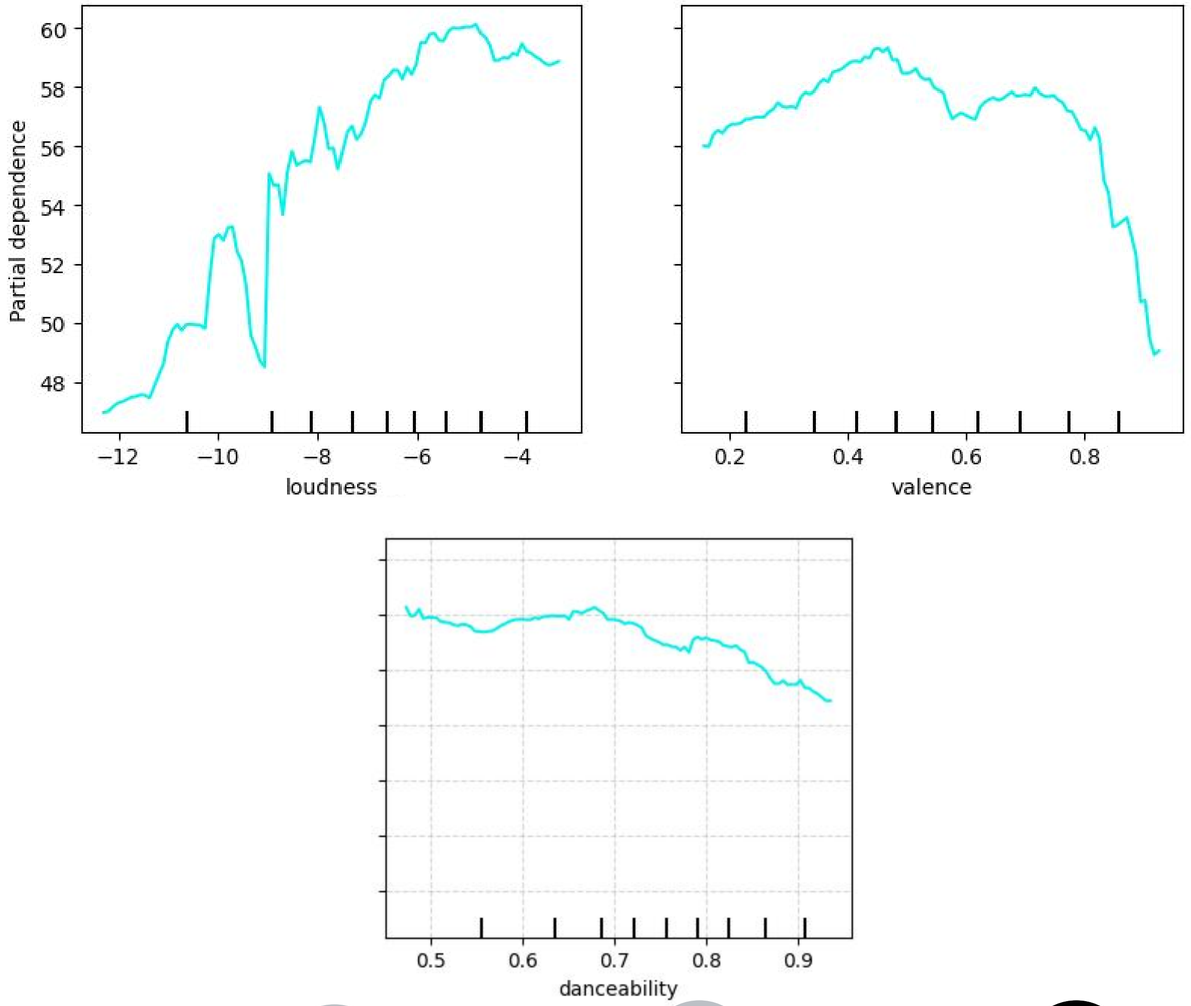
R2: 0.548

Random Tree



Topic | Dataset | Exploration | Limitations | Insights

Partial Dependency Plots



Positive Correlation:
Loudness & Popularity

Nonlinear Correlation:
Valence rises then flattens.
Danceability rises then flattens.

Model Testing

Justin Bieber - Peaches
PI: 99 ; Predicted: 98.94
BTS - Butter
PI: 99 ; Predicted: 98.566
Doja Cat - Kiss Me More (feat. SZA)
PI: 98 ; Predicted: 99.51
Bruno Mars - Leave the Door Open
PI: 96 ; Predicted: 95.4
Olivia Rodrigo - good 4 u
PI: 97 ; Predicted: 95.99

Observations

- All songs with a PI of 96–99 were predicted within ~1.5 points --> precision in popular songs.
- Mainstream genres (pop, R&B, rap) are modeled effectively
- Popular artists were predicted accurately --> model better understands well-represented styles.
- High PI songs are slightly underestimated
→ good 4 u, Butter, and Peaches all had predicted scores slightly below their actual PI --> conservative bias in the model's upper-end predictions.
- Exception: One high-PI song was overestimated
→ Kiss Me More was predicted higher than its actual PI (98 → 99.51), showing the model may sometimes overweight certain upbeat or highly danceable features.
- Regardless of artist or vocal style, the model maintained consistency --> popularity predictions were driven by audio features, not artist bias.

Limitations

Popularity Index

- Based on engagement, but lacks full transparency
- Doesn’t account for visual trends or memes

TikTok

- Algorithm and virality influenced by influencers, trends, and video edits
- Song popularity isn't purely audio-driven

Dataset

- Language
- Features are programically curated
- Song popularity isn't purely audio-driven

Variables

- No metadata on user interactions, context, or challenge use
- Audio features are simplified and subjective

Insights & Uses



- Loudness, valence, and duration are top predictors
- Popular songs show higher model accuracy
- Shorter songs with upbeat tones are more likely to go viral
- Overprediction is more common on mid-tier tracks

- Help musicians design songs with viral potential
- Assist marketers in identifying trends early
- Guide A&R teams in scouting tracks likely to blow up
- Support researchers in studying music + social media dynamics

Thank You



Questions?

Appendix: Heatmap

Creating a Heatmap

```
[ ]  columns_to_drop = ['key', 'loudness', 'mode', 'speechiness', 'instrumentalness', 'liveness', 'duration']
     existing_columns_to_drop = [col for col in columns_to_drop if col in tiktok.columns]

     # Drop only the ones that exist
     tiktok_numeric = tiktok.select_dtypes(include=['float64', 'int64']).drop(columns=existing_columns_to_drop)

     # Now generate correlation matrix and plot
     import seaborn as sns
     import matplotlib.pyplot as plt

     corr_matrix = tiktok_numeric.corr()

     plt.figure(figsize=(12, 10))
     sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
     plt.title('Correlation Heatmap')
     plt.show()
```

Appendix: Regression

```
slope = model.coef_[0]
intercept = model.intercept_

print(f"The regression equation is: y = {slope:.2f}x + {intercept:.2f}")

plt.figure(figsize=(8, 6))

plt.scatter(X_train, y_train, color='blue', label='Training Data')

plt.plot(X_train, model.predict(X_train), color='red', label=f'Regression Line: y = {slope:.2f}x + {intercept:.2f}')

plt.xlabel('duration_mins')
plt.ylabel('popularity')
plt.title('Linear Regression on Training Set')
plt.legend()

plt.show()

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error (MSE): {mse:.2f}')
print(f'R-squared (R²): {r2:.2f}')
```

```
plt.figure(figsize=(8, 6))

plt.scatter(y_test, y_pred, color='blue', label='Training Data')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='dashed', label="Perfect Fit")
plt.xlabel("loudness , valence")
plt.ylabel("popularity")
plt.title("Actual vs. Predicted Popularity")
plt.legend()

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error (MSE): {mse:.2f}')
print(f'R-squared (R²): {r2:.2f}')
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
import numpy as np

# Keep only numeric columns
X = df.drop(columns=['popularity'])  # first drop 'popularity'
X = X.select_dtypes(include=['int64', 'float64'])  # keep only numbers
y = df['popularity']

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predict
y_pred = rf_model.predict(X_test)

# R2 score
r2 = r2_score(y_test, y_pred)
print(f"Random Forest R² Score: {r2:.3f}")

# ---- FEATURE IMPORTANCE ----
importances = rf_model.feature_importances_
feature_names = X.columns
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(12, 6))
plt.title('Feature Importances for Predicting Popularity')
```

```
plt.bar(range(X.shape[1]), importances[indices], align='center')
plt.xticks(range(X.shape[1]), [feature_names[i] for i in indices], rotation=45, ha='right')
plt.tight_layout()
plt.show()

# Print them ranked
print("\nFeature Importances Ranked:")
for f in range(X.shape[1]):
    print(f"{f + 1}. {feature_names[indices[f]]}: {importances[indices[f]]:.4f}")

# ---- ACTUAL vs PREDICTED ----
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='#00F2EA', alpha=0.5, edgecolor='black', label='Predicted Popularity')  # teal points
plt.plot([0, 100], [0, 100], color='#FF2C92', linestyle='--', linewidth=2, label='Perfect Fit')  # hot pink line
plt.xlabel('Actual Popularity (y_test)')
plt.ylabel('Predicted Popularity (y_pred)')
plt.title('Random Forest Model: Actual vs Predicted Popularity')
plt.xlim(0, 100)
plt.ylim(0, 100)
plt.grid(True, linestyle='--', alpha=0.5)
plt.style.use('default')  # white background

plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='#00F2EA', edgecolor='black', alpha=0.5, label='Predicted Popularity')
plt.plot([0, 100], [0, 100], color='#FF2C92', linestyle='--', linewidth=2, label='Perfect Fit')
plt.xlabel('Actual Popularity (y_test)')
plt.ylabel('Predicted Popularity (y_pred)')
plt.title('Random Forest Model: Actual vs Predicted Popularity')
plt.xlim(0, 100)
plt.ylim(0, 100)
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

Appendix: PDP

```
from sklearn.inspection import PartialDependenceDisplay
import matplotlib.pyplot as plt

# rf_model (fitted model), X (your feature matrix), feature names

features_to_plot = ['loudness', 'valence', 'danceability']

# Plot PDPs
fig, ax = plt.subplots(figsize=(12, 4))  # wider layout
PartialDependenceDisplay.from_estimator(
    rf_model,
    X,
    features=features_to_plot,
    feature_names=X.columns,
    ax=ax,
    kind='average',
    line_kw={'color': '#00F2EA'},  # Teal TikTok color
)

plt.suptitle('Partial Dependency Plots (PDP) – Random Forest Model', fontsize=14)
plt.tight_layout()
plt.subplots_adjust(top=0.85)  # space for title
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Appendix: Dataset

https://www.kaggle.com/datasets/yamqwe/tiktok-trending-tracks?resource=download